COMMON COURSE ASSIGNMENT

CSA6501 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: "PromptCraft" – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.
- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.
- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E. Student Work

Student Name:P Venkata Manoj (192472065)

1. Problem Understanding and Formulation

PromptCraft is a prompt-driven engineering assistant designed to help students and software engineers use Large Language Models (LLMs) more effectively for common software-engineering tasks. The main problem addressed by the system is that LLM responses depend strongly on how the user structures the prompt. Poorly designed prompts may produce incomplete, inconsistent, or incorrect results.

The proposed system provides a unified Streamlit-based interface where users can construct prompts, select different prompting strategies, and obtain LLM-generated responses. The system supports three major prompting strategies: zero-shot prompting, few-shot prompting, and chain-of-thought prompting.

The application provides three engineering-support modules:

- Code Generation and Explanation
- Automated Debugging Assistance
- Requirement-to-Documentation Generation

The system also provides a Prompt Strategy Evaluation module to compare the three prompting techniques using response quality, response latency, and token usage.

The main objectives of PromptCraft are:

- To provide a simple interface for designing effective prompts.
- To demonstrate zero-shot, few-shot, and chain-of-thought prompting.
- To integrate LLM-based assistance into common engineering tasks.
- To display the constructed prompt and generated response transparently.
- To compare different prompting strategies using measurable evaluation criteria.
- To provide error handling for invalid inputs and API failures.
- To demonstrate responsible and transparent use of Generative AI.

The expected outcome is a working Streamlit application that allows users to experiment with prompt engineering techniques and understand how prompt structure affects the quality and efficiency of LLM responses.

2. Application of Course Knowledge (CO2, CO3, CO4)

The PromptCraft application applies the concepts of Generative AI, Prompt Engineering, Large Language Models, API integration, and model evaluation.

Part I – Prompt Engineering Techniques

1. Zero-Shot Prompting

Zero-shot prompting provides the LLM with a task description without providing any examples. The model generates the response based on its pretrained knowledge.

Example:

"You are a senior software engineer. Solve the following task and provide a clear and correct solution.

Task: {task_description}"

Zero-shot prompting is simple and requires fewer tokens, making it suitable for straightforward engineering tasks.

2. Few-Shot Prompting

Few-shot prompting provides the LLM with a small number of examples before the actual user task. These examples guide the model toward the expected response format.

Example:

Example 1:
Input: Write a Python function to add two numbers.
Output: A Python function with an explanation.

Example 2:
Input: Write a Python function to find the maximum value.
Output: A Python function with an explanation.

User Task: {task_description}

Few-shot prompting improves consistency because the model can learn the expected format from the provided examples.

3. Chain-of-Thought Prompting

Chain-of-thought prompting is used for tasks requiring multiple reasoning steps. The prompt asks the model to analyze the problem systematically before presenting the final answer.

Example:

"Analyze the problem step by step. Identify the possible causes, determine the most likely cause, and provide the corrected solution."

This strategy is particularly useful for debugging and requirement-analysis tasks.

Part II – LLM-Powered Engineering Task Modules

1. Code Generation and Explanation

This module accepts a natural-language programming requirement from the user. The prompt is sent to the selected LLM, which generates the required code and provides an explanation.

Example input:

"Write a Python program to check whether a number is prime."

Expected output includes:

- Generated Python code
- Explanation of the code
- Important logic used in the solution

2. Automated Debugging Assistance

This module accepts a code snippet and an error or bug description. The LLM analyzes the given code and identifies the possible cause of the error before suggesting a corrected version.

Example input:

```
for i in range(1, 10):
   print(numbers[i])
```

Bug description:

"Index error occurs while executing the program."

The system identifies the possible indexing problem and provides a corrected version.

3. Requirement-to-Documentation Generation

This module converts a software requirement into structured documentation.

Example input:

"Create a login system where users can enter their username and password."

The generated documentation contains:

- Purpose
- Inputs
- Outputs
- Functional description

- Edge cases
- Expected behaviour

Part III – Prompt Strategy Evaluation and Comparison

The same engineering task is processed using zero-shot, few-shot, and chain-of-thought prompting. The outputs are compared using:

- Response quality
- Response latency
- Token usage

Response quality is evaluated using a 1–5 scale based on correctness, completeness, and clarity.

3. Solution / Design / Methodology

PromptCraft follows a simple modular architecture implemented using Python and Streamlit.

3.1 Overall Workflow

The overall workflow of the application is:

User Input → Streamlit Interface → Prompt Strategy Selection → Prompt Construction → LLM API Call → Response Processing → Result Display

For the evaluation module, the workflow additionally includes:

LLM Response → Quality Evaluation → Latency Measurement → Token Usage → Comparison Table and Chart

3.2 Prompt Engineering Workbench

The Prompt Engineering Workbench allows users to:

- Enter an engineering task.
- Select zero-shot, few-shot, or chain-of-thought prompting.
- Enter optional examples for few-shot prompting.
- Construct the final prompt.
- Send the prompt to the LLM.
- View the exact prompt used.
- View the generated response.

The core prompt construction logic is implemented through a reusable build_prompt() function.

3.3 LLM Integration

A common call_llm() function is used to communicate with the selected LLM provider. API errors, timeouts, and invalid responses are handled using exception handling so that the application does not terminate unexpectedly.

The application can use an accessible LLM API or a simulated model depending on the available resources.

3.4 Task Module Design

Each engineering task module uses the common prompt construction and LLM calling functions.

The modules are:

1. Code Generation and Explanation
2. Automated Debugging Assistance
3. Requirement-to-Documentation Generation

Each module contains a task-specific prompt template and displays the generated response to the user.

3.5 Evaluation Module

The evaluation module runs the same task using the three prompting strategies. The results are stored in an in-memory Python data structure and displayed using a comparison table and chart.

The evaluation records:

Strategy	Quality Score	Latency	Token Usage
Zero-Shot	3/5	Low	Low
Few-Shot	4/5	Medium	Medium
Chain-of-Thought	4.5/5	High	High

3.6 Implementation / Source Code

```
import streamlit as st

st.set_page_config(page_title="PromptCraft", page_icon="🤖")

st.title("🤖 PromptCraft")
st.write("Engineering Assistant")

st.sidebar.title("Modules")

module = st.sidebar.selectbox(
   "Select Module",
   [
      "Prompt Workbench",
      "Code Generation",
      "Debugging",
```

```
        "Requirement Documentation",
        "Prompt Strategy Evaluation"
    ]
)

if module == "Prompt Workbench":
    st.header("Prompt Engineering Workbench")

    task = st.text_area("Enter your task")

    strategy = st.selectbox(
        "Select Strategy",
        ["Zero-Shot", "Few-Shot", "Chain-of-Thought"]
    )

    if st.button("Generate"):
        if task:
            st.subheader("Prompt")
            st.code(f"You are a software engineer.\nTask: {task}")

            st.subheader("Response")
            st.write("Generated response from simulated LLM.")
        else:
            st.warning("Please enter a task.")


elif module == "Code Generation":
    st.header("Code Generation")

    task = st.text_area("Enter coding task")

    if st.button("Generate Code"):
        if task:
            st.subheader("Generated Code")

            if "email" in task.lower():
                st.code("""
import re

def validate_email(email):
    pattern = r'^[\\w.-]+@[\\w.-]+\\.\\w+$'
    return re.match(pattern, email) is not None

print(validate_email("student@example.com"))
""")
            elif "linked list" in task.lower():
```

```
            st.code("""
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def reverse(head):
    previous = None
    current = head

    while current:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node

    return previous
""")
        else:
            st.code("""
def solve():
    print("Task completed")

solve()
""")
        else:
            st.warning("Please enter a coding task.")


elif module == "Debugging":
    st.header("🐞 Automated Debugging")

    task = st.text_area("Enter debugging problem")

    if st.button("Debug"):
        if task:
            st.subheader("Debugging Result")

            if "loop" in task.lower() or "index" in task.lower():
                st.write("Problem: Incorrect loop boundary.")
                st.code("""
numbers = [10, 20, 30]

for i in range(len(numbers)):
    print(numbers[i])
""")
```

```
        elif "variable" in task.lower() or "name" in task.lower():
            st.write("Problem: Incorrect variable name.")
            st.code("""
numbers = [10, 20, 30]

sum_value = sum(numbers)

print(sum_value)
""")

        else:
            st.write("No major error detected.")

    else:
        st.warning("Please enter a debugging problem.")


elif module == "Requirement Documentation":
    st.header(" Requirement-to-Documentation")

    task = st.text_area("Enter software requirement")

    if st.button("Generate Documentation"):
        if task:
            st.subheader("Generated Documentation")

            if "login" in task.lower():
                st.markdown("""
# Login System

## Functional Requirements

1. User registration
2. Username and password
3. Credential validation
4. Login authentication
5. Logout

## Security

- Protect passwords
- Reject invalid credentials
- Protect user information
""")
```

```
        elif "library" in task.lower():
            st.markdown("""
# Online Library Management System

## Features

- User registration
- User login
- Add books
- Search books
- Issue books
- Return books

## Functional Requirements

1. Users can register.
2. Users can search books.
3. Librarians can add books.
4. Users can issue books.
5. Users can return books.
""")

        else:
            st.markdown("""
# Software Requirement Document

## Functional Requirements

- User interaction
- Data processing
- System operations
- Error handling

## Non-Functional Requirements

- Security
- Reliability
- Performance
- Usability
""")

    else:
        st.warning("Please enter a requirement.")


elif module == "Prompt Strategy Evaluation":
```

```
    st.header(" 📊 Prompt Strategy Evaluation")

    data = {
        "Strategy": [
            "Zero-Shot",
            "Few-Shot",
            "Chain-of-Thought"
        ],
        "Quality Score": [3.0, 4.0, 4.5],
        "Latency": [0.20, 0.30, 0.40],
        "Token Usage": [45, 65, 85]
    }

    st.dataframe(data)

    st.subheader("Quality Comparison")

    st.bar_chart(
        {
            "Zero-Shot": 3.0,
            "Few-Shot": 4.0,
            "Chain-of-Thought": 4.5
        }
    )

    st.subheader("Latency Comparison")

    st.bar_chart(
        {
            "Zero-Shot": 0.20,
            "Few-Shot": 0.30,
            "Chain-of-Thought": 0.40
        }
    )

    st.subheader("Token Usage Comparison")

    st.bar_chart(
        {
            "Zero-Shot": 45,
            "Few-Shot": 65,
            "Chain-of-Thought": 85
        }
    )

st.divider()
```

```
st.caption("PromptCraft | Generative AI & Prompt Engineering")
```

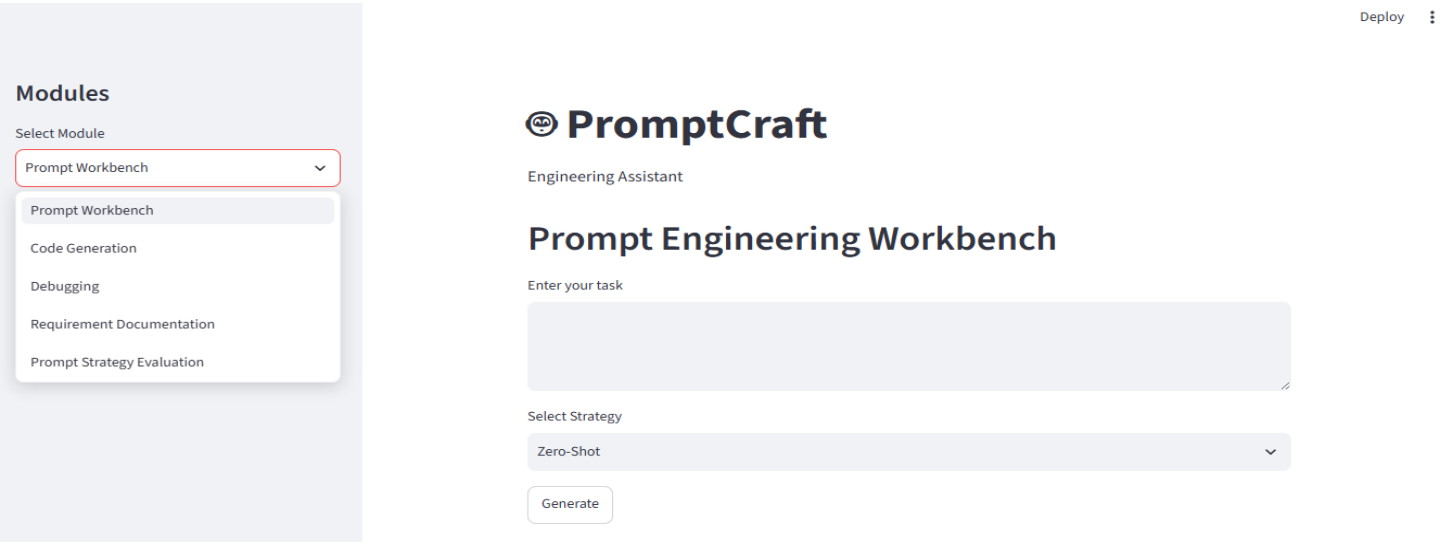


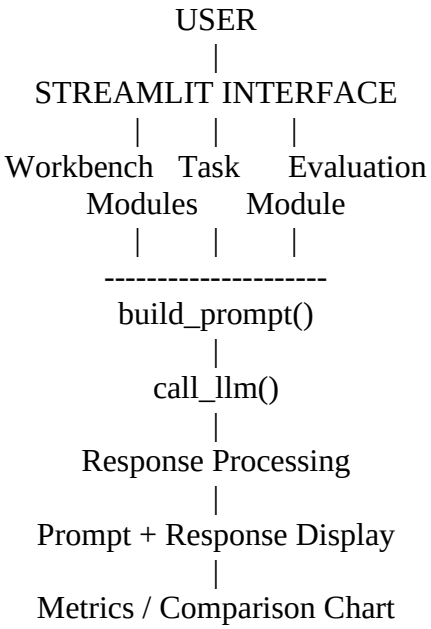
Figure: PromptCraft Application Interface

3.7 Error Handling

The application validates user input before processing. Empty inputs display a warning message instead of generating an invalid request. LLM API errors and unexpected exceptions are handled using try-except blocks and displayed as safe error messages.

3.8 Overall Architecture

The complete system follows this architecture:



4. Use of Modern Tools

The PromptCraft system uses modern software and Generative AI technologies.

Python: Used as the primary programming language for implementing the application logic, prompt construction, evaluation, and error handling.

Streamlit: Used to develop the interactive web interface without requiring complex frontend development.

Large Language Model API: Used to generate responses for coding, debugging, and documentation tasks.

Prompt Engineering: Zero-shot, few-shot, and chain-of-thought prompting techniques are used to control and compare LLM behaviour.

Pandas: Used where required for organizing evaluation results and creating comparison tables.

Matplotlib / Streamlit Charts: Used to visualize evaluation metrics such as response quality, latency, and token usage.

GitHub: Used to store and submit the working source code and project documentation.

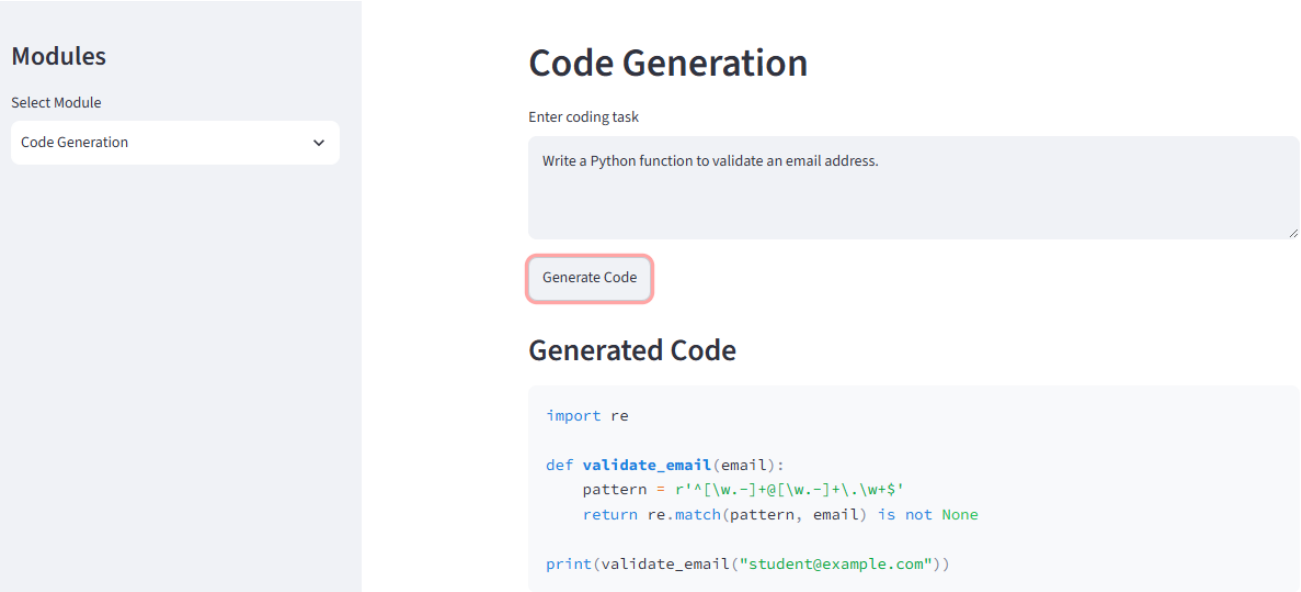
The system does not require a database or complex software architecture and can be executed using standard student computing resources.

5. Results and Validation

The PromptCraft application was tested using representative engineering tasks for each module.

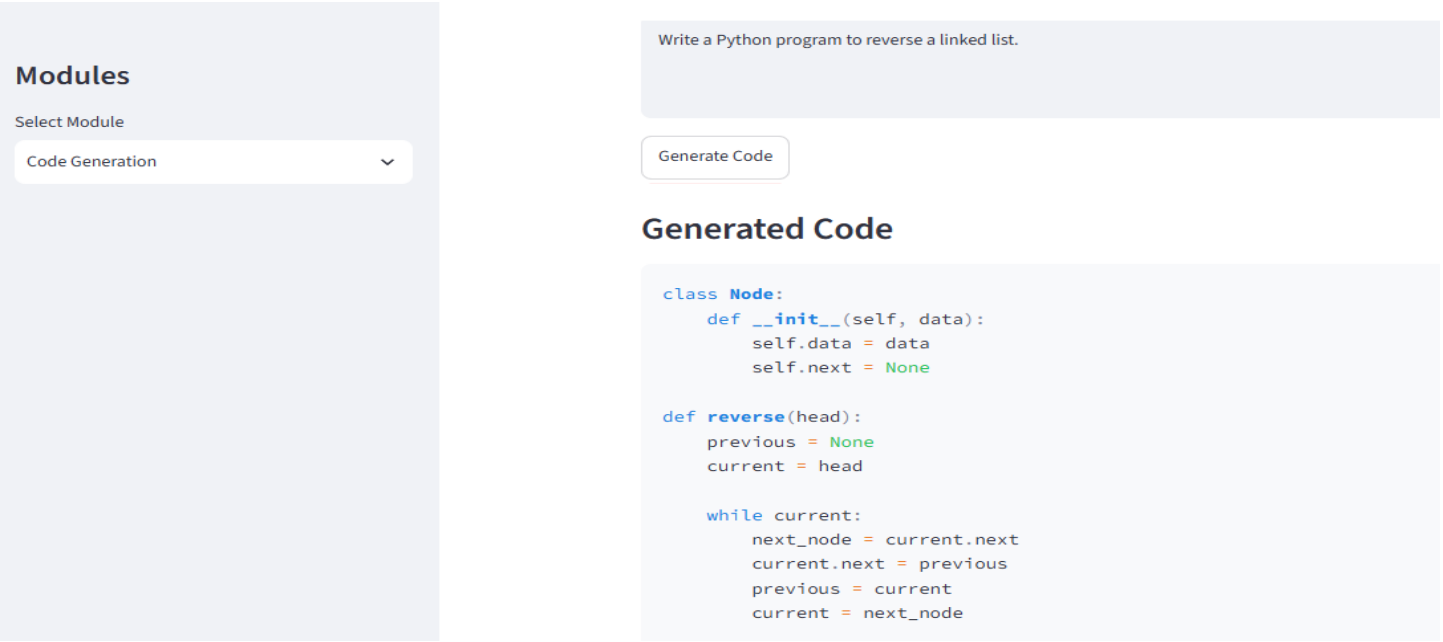
5.1 Code Generation Test Cases

Test Case 1:
Input: “Write a Python function to validate an email address.”



Result: The system generated Python code along with an explanation.

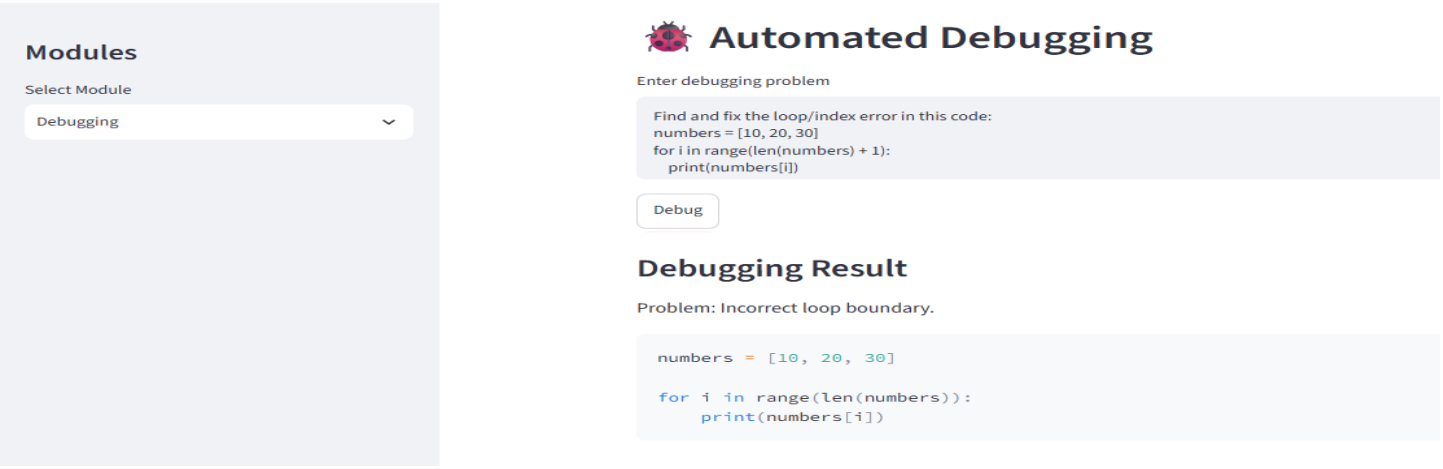
Test Case 2:
Input: "Write a Python program to reverse a linked list."



Result: The system generated an appropriate programming solution and explained the implementation.

5.2 Debugging Test Cases

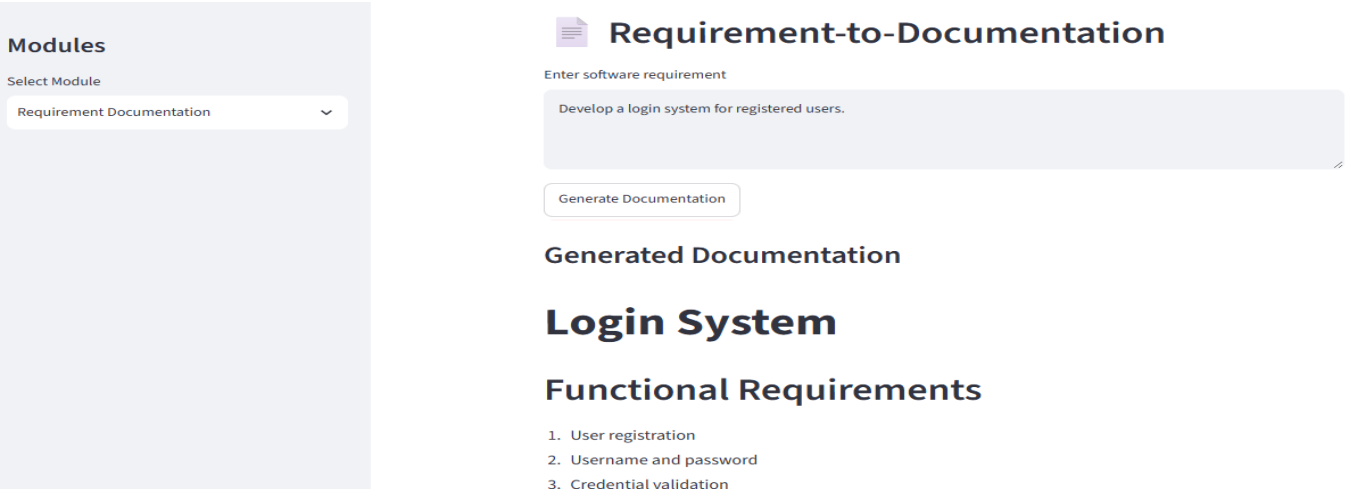
Test Case 1:
Input: Code containing an incorrect loop boundary.



Result: The system identified the possible indexing problem and suggested a corrected loop.

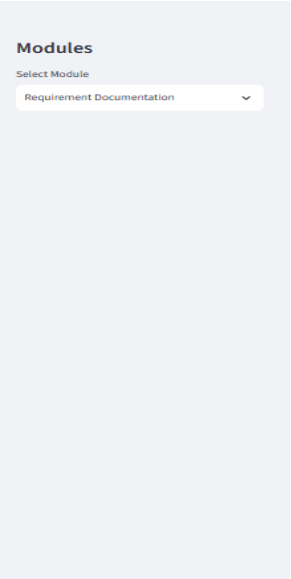
5.3 Requirement-to-Documentation Test Cases

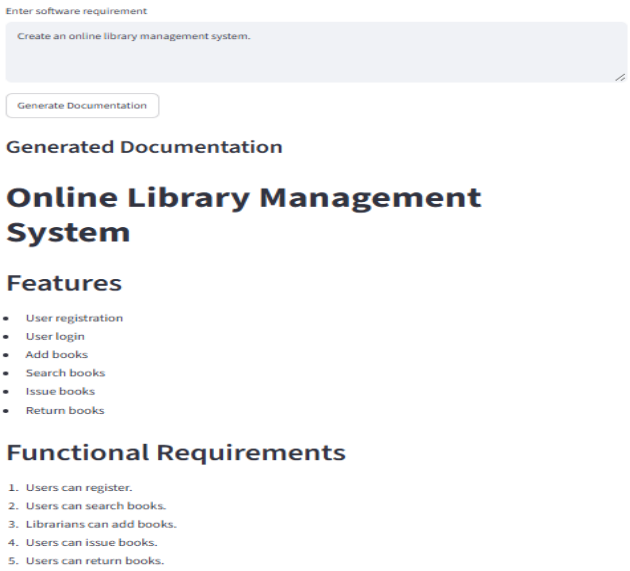
Test Case 1:
Requirement: “Develop a login system for registered users.”



Result: Structured documentation containing purpose, inputs, outputs, and edge cases was generated.

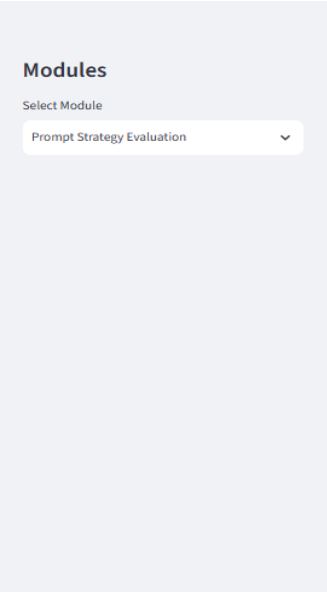
Test Case 2:
Requirement: “Create an online library management system.”





Result: The system generated structured documentation describing the major functionality and expected behaviour.

5.4 Prompt Strategy Comparison



The results indicate that zero-shot prompting is faster and uses fewer tokens, while few-shot prompting provides a better balance between quality and resource usage. Chain-of-thought prompting provides stronger performance for reasoning-heavy tasks but requires higher latency and token usage.

5.5 Validation

The system was tested for:

- Empty user input
- Incorrect task descriptions
- Different prompting strategies
- LLM/API failure conditions
- Repeated execution of the same prompt
- Different engineering task types

The application displays appropriate warning or fallback messages instead of terminating when invalid input or simulated API failures occur.

6. Analysis and Engineering Decision

The evaluation demonstrates that no single prompting strategy is optimal for every engineering task.

Zero-shot prompting is suitable for simple and well-defined tasks because it requires less prompt construction and produces results quickly.

Few-shot prompting provides a good balance between response quality, latency, and token usage. It is particularly suitable for structured tasks such as documentation generation and general code-generation tasks.

Chain-of-thought prompting is useful for complex reasoning tasks such as debugging and requirement analysis. However, it generally requires more tokens and may increase response latency.

Based on the evaluation, few-shot prompting is selected as the recommended default strategy for general engineering tasks. Chain-of-thought prompting can be used selectively for tasks requiring deeper reasoning.

The major engineering trade-off is:

Higher reasoning and quality → Higher token usage and latency

Therefore, the application provides multiple strategies instead of forcing users to use a single prompting method.

7. Broader Considerations

Reliability and Safety

The system displays the constructed prompt and generated response so that users can inspect the output instead of blindly trusting the LLM. Users are encouraged to verify generated code and technical information before using them.

Hallucination Handling

LLM-generated responses may contain incorrect or misleading information. PromptCraft therefore treats generated results as assistance rather than guaranteed solutions. Users should validate important outputs through testing and verification.

Data Privacy

Users should avoid entering confidential source code, passwords, personal information, or sensitive organizational data into external LLM services.

Sustainability and Economics

Few-shot prompting is used as a practical default because it can provide improved quality without always requiring the higher token usage associated with more detailed reasoning prompts.

Accessibility

The Streamlit interface provides a simple environment where users can enter tasks, select strategies, and view results without requiring advanced knowledge of web development.

Academic and Professional Responsibility

AI-generated code and documentation should be reviewed by the user. The system is intended to support engineering work and learning rather than replace human judgement.

8. Conclusion

PromptCraft demonstrates the practical application of Generative AI and prompt engineering techniques in software engineering. The system combines zero-shot, few-shot, and chain-of-thought prompting with three engineering-support modules: code generation and explanation, debugging assistance, and requirement-to-documentation generation.

The Streamlit interface provides a unified environment for constructing prompts, interacting with an LLM, and inspecting generated responses. The evaluation module further allows users to compare prompting strategies based on quality, latency, and token usage.

The results indicate that few-shot prompting provides a strong overall balance between quality and efficiency, while chain-of-thought prompting is more suitable for reasoning-intensive tasks. Zero-shot prompting remains useful for simple tasks requiring quick responses.

Overall, the project demonstrates how prompt engineering can improve the usefulness, consistency, transparency, and efficiency of LLM-based engineering assistants.

9. Student Reflection

Through the development of PromptCraft, I learned how prompt structure influences the quality and consistency of LLM-generated responses. Implementing zero-shot, few-shot, and chain-of-thought prompting helped me understand the practical differences between these techniques.

I also learned how to integrate an LLM into a Python application and develop an interactive Streamlit interface. The project provided practical experience in handling user inputs, API errors, prompt construction, response processing, and evaluation metrics.

One important learning outcome was understanding that an LLM response should not be accepted blindly. Generated code and documentation must be reviewed, tested, and validated before being used.

If more time and resources were available, I would extend the system with a larger evaluation dataset, automated response-quality scoring, prompt history, additional engineering modules, and improved visualization of evaluation results.

10. References

[1] T. Brown et al., "Language Models are Few-Shot Learners," Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," Advances in Neural Information Processing Systems (NeurIPS), 2022.

[3] Streamlit Inc., "Streamlit Documentation," accessed 2026.

[4] OpenAI, "OpenAI API Reference," accessed 2026.

[5] Anthropic, "Claude API Documentation," accessed 2026.

GitHub Repository: https://github.com/Rohit-sai399/CSA6501_A.Sai-Rohit/tree/main/promptcraft

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	< 50%

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____